CE152: Object-Oriented Programming

Methods

Dr. Sebastian Halder

CSEE

2023

Refresher on Objects

Defining a class

- Before we can use our own Objects in our code we must define them as a class
- This creates a class called MyClass

```
class MyClass{

}
```

- We could instantiate it using the standard constructor and the `new` keyword

```
// MyClass() is the constructor
MyClass myClass1 = new MyClass();
```

Class variables

- We add the member variables inside the body (defined by the $\{\}$) of the class
- These variables contain the data of the class

```
class MyClass {
    int myInt;
    String myString;
}
```

- If we want to add functionality to the class we need to define methods

How can we call and define methods?

Declaring a method

- When declaring a method we need:
  - The **return type** (can be `void` or any primitive or Object)
  - The **name** (e.g. `myMethod1`)
  - The **parameters** (in the (), can be blank or any number of primitives or Objects)
  - The **body** in {} that will contain the code that is executed when we call the method

```
void myMethod1(){

}
```

Adding a return type

- What the method returns defines how the code we call the method from is informed about the outcome
- Some methods do not return anything as in the previous example
- These methods are `void` and do not need a `return` statement
- If we want the method to return something we replace the `void` with the data type (not the variable name!)
- In the example below `myMethod1` will now return an `int` (a primitive that holds an integer/whole number)
- For any return type except `void` we need to add a `return` statement

```
int myMethod1(){
  return 42;
}
```

Adding parameters

- The parameters are placed in the () following the method name
- They are defined with their type and a variable name
- Multiple parameters are separated by commas
- The content is defined when the method is called
- The method below will now receive two `int` as parameters and return the sum

```
int myMethod1(int x, int y){
  return x+y;
}
```

Example

```
int myMethod1(int x){
  int y = x * 2;
  return y;
}
```

Adding the method to the class

- Java allows us to add methods anyway in the body of the class
- According to convention we should add methods after the class variables and the constructor

```
class MyClass {
    // 1. Variables
    // 2. Constructors
    // 3. Methods e.g.:
    int myMethod1(int x, int y){
      return x+y;
    }
}
```

Calling the method

- If we want to call the method we need to create the `Object` first

```
MyClass mc1 = new MyClass();
mc1.myMethod1(15,2);
```

- A special type of method, called `static`, can be called without creating the `Object`, but more on that later

The return value

- In the previous example we ignored the return value
- This does not cause an error but usually we want to store the value
- We can do this by assigning it to a variable

```
int x = mc1.myMethod1(15,2);
```

- We can also perform operations with the return value

```
int x = 151 + mc1.myMethod1(15,2);
```

- Can you calculate the value of x?

Using class variables in a method

- If we have variables in our class we can simply use them in our methods

```
class MyClass {
    // 1. Variables
    int myClassInt = 5;
    // 2. Constructors
    // 3. Methods e.g.:
    int myMethod1(int x, int y){
      return myClassInt+x+y;
    }
}
```

Initialising the class variables

- We can initialise the class variables in the constructor

```
class MyClass {
    // 1. Variables
    int myClassInt = 5;
    // 2. Constructors
    MyClass(int myClassInt){
      this.myClassInt = myClassInt
    }
    // 3. Methods e.g.:
    int myMethod1(int x, int y){
      return myClassInt+x+y;
    }
}
```

The this keyword

```
MyClass(int myClassInt){
  this.myClassInt = myClassInt
}
```

- Remember: with the `this` keyword we specify the class variables
- In the constructor:
  - `myClassInt` refers to the parameter
  - `this.myClassInt` refers to the class variable
  - We do not have to use the same name for the parameter but doing so makes the purpose of the parameter known

A method cannot be a constructor and vice versa

```
public Circle(double radius){
  this.radius = radius;
}

public int getRadius(){
  return radius;
}
```

- A constructor *cannot* have a return type, a method *must* have one even if it is `void`
- A constructor is invoked using `new`, a method using '.'
- A constructor *must* have the name of the class, a method can have any name (even that of the class)

Some additional terms

- We call what is between the $\{\}$ of the method the *body* of the method
- Variables we create inside the method *body* exist only there and are deleted when we leave the method
- They variables inside the *body* are *local variables*
- We may also refer to the method body as a *block*

Live-coding

- Let's define a class for some simple mathematical operations

Summary

The class template

- It is ***very important*** that you understand the basic structure of Java classes exemplified below!

```
class MyClass {
    // 1. Variables
    int myClassInt = 5;
    // 2. Constructors
    MyClass(int myClassInt){
      this.myClassInt = myClassInt
    }
    // 3. Methods e.g.:
    int myMethod1(int x, int y){
      return myClassInt+x+y;
    }
}
```

Thank you and see you in the next video!